```
CREATE table workdata (date text, empname text, projname
text, custname text, hours int)
```

To insert data records, let's use the following INSERT SQL statement:

```
INSERT INTO workdata VALUES('01/01/2015', 'john',
'projectA','customerA',8)
```

To query records for projectA, we could execute the following SELECT SQL statement:

```
SELECT * FROM workdata WHERE projname = 'projectA'
```

To remove records for projectA, we could execute the following DELETE SQL statement:

```
DELETE FROM workdata WHERE projname = 'projectA'
```

Note: If you omit the WHERE clause in the DELETE FROM statement, it will delete all records in the table.

An introduction to SQLite3 by creating and using a table with data

It is common to find an application, which works with data, to be designed with a data layer that works with a database system, most commonly an RDBMS. An industry level RDBMS, though, is a big beast to handle, with its own server and hardware requirements. A database server also requires some dedicated set-up and management expertise. Most applications, though, do not need such an extensive database system to work with. In the light of this, it is nice to note that Python comes with package SQLite3 with the default installation from Python 2.5. It conforms to a non-standard SQL version as specified in Python DB-API 2.0. The following is quoted verbatim from the SQLite3 documentation:

"SQLite is a C library that provides a lightweight disk-based database that doesn't require a separate server process and allows accessing the database using a non-standard variant of the SQL query language. Some applications can use SQLite for internal data storage. It's also possible to prototype an application using SQLite and then port the code to a larger database such as PostgreSQL or Oracle."

Interfaces available with SQLite3

1. *sqlite3.connect(dbname):* The connect call creates a database connection to the database file specified by the *dbname* parameter and returns an object of type *Connection*. The *dbname* could also use the special value of ":memory:" to create an in-memory database, though the database ceases to exist once the connection is closed.
2. *Connection.cursor():* This call helps create a *Cursor* object for the database connection. The Cursor object is the Python abstraction for creating multiple execution environments through the same database connection. *cursors* provide interfaces for SQL execution and parsing of results.
3. *Cursor.execute(sql)*: This interface helps execute a single SQL statement as specified in the string parameter *sql*. There are other interfaces available to execute many SQL statements: *executemany,* as also ways to execute a script: *executescript*
4. *Cursor.fetchone():* If preceded by a query execution, this interface returns a single sequence of the result set, as a tuple by default, or *None* object if no more results exist. So if you have multiple records returned from a query and would like to process them one by one, you would need to call this many times till a *None* record is returned. There are other interfaces like *fetchmany(size)* to fetch a list of records of a specified size. Also, *fetchall()* fetches all records in the same call, and returns a complete list of records from the query execution.
5. *Connection.commit():* This interface commits the changes to the database. If there were DDL or DML statements executed, they need to be committed to the database for the changes to persist.
6. *sqlite3.Row:* This type can be used as a *row_factory* instance for the connection object. By default, the results of a query execution are handled as a tuple with indexed access to the attributes or values of the tuple. Using a *row_factory,* one can make custom access of the values using attribute names for mapping, which are lot more meaningful. While SQLite3 allows you to define your custom *row_factory* mapping, I strongly recommend that you use the built-in *sqlite3.Row* to access the results conveniently using column names.

Let us look at the above interfaces at work using the Python command line. In the first segment below, we connect to a *test.db* as a database file and open a cursor to the same connection.

```
>>> import sqlite3
>>> conn = sqlite3.connect('test.db')
>>> c=conn.cursor()
```

We then create the table of interest and insert one record for *projectA* for employee *john* and commit the changes.

```
>>> c.execute('''CREATE table workdata (date text, empname text,
projname text,
custname text, hours int)
... ''')
<sqlite3.Cursor object at 0x00000000021C1B20>
>>> c.execute("INSERT INTO workdata VALUES('01/01/2015', 'john',
'projectA','cus
tomerA',8)")
<sqlite3.Cursor object at 0x00000000021C1B20>
>>> conn.commit()
```

We can then query for records of interest using a *select* query. We can see that the *fetchone()* call returns a record tuple, by default.